Recalibrate After You Accelerate: Few-Step Flow Sampling Silently Disarms a VLA's Failure Monitors

Anonymous Author(s)
Affiliation
Address
email

Abstract: Failure monitors for vision-language-action (VLA) policies are calibrated on the policy as trained, but the deployed policy is usually accelerated. We freeze four monitors on SmolVLA, calibrate their alarms on reference rollouts, and run compressed policies closed loop on LIBERO-Long. Sampling the flow-matching action expert with 2 Euler steps instead of 10 cuts policy latency $2.5\times$ at unchanged task success. It also shrinks action-chunk disagreement by a uniform $0.42\times$, so an action-consistency monitor that caught 49% of failures catches none of 68, and an expert-state probe loses a quarter of its detections. Monitors that read the vision-language backbone, which step reduction leaves untouched, are unaffected. Because conformal thresholds calibrated on successes bound only false alarms, the broken monitors pass a false-alarm check. Weight quantization shows the mirror image: it moves a backbone probe toward "looks safe" on identical inputs (-0.3, -2.3 and -4.6 SD at int8 and two int4 settings), while its detection falls from 0.67 to 0.54, 0.09 and 0.

Keywords: VLA, acceleration, quantization, runtime monitoring, conformal prediction

1 Introduction

A VLA policy that reaches a robot has usually been changed twice:

- it has been **accelerated**, by quantization, pruning or fewer denoising steps [1, 2, 3];
- it has been wrapped in a runtime **failure monitor**, whose alarm threshold is calibrated on rollouts of the policy as trained [4, 5, 6, 7].

The accelerated policy inherits the monitor. Acceleration papers check that task success holds, but a conformal threshold keeps its guarantee only while calibration and deployment scores are exchangeable [8], and acceleration can break that without moving success. For language-model probes, updates can preserve ranking while moving scores [9]. Concurrent work shows the same for a SmolVLA probe under simulated int4, replaying the full-precision policy's observations open loop [10].

We test the deployed configuration: compressed policies run closed loop, with four frozen monitors from different families. Step reduction disarms the action-side monitors and leaves the backbone monitors intact, at unchanged task success. Calibrating on successes hides the failure: when scores shift downward, the false-alarm rate improves while detection collapses. Weight quantization does the reverse, pushing a backbone probe toward "safe".

Submitted to the 10th Conference on Robot Learning (CoRL 2026). Do not distribute.

2 Setup

Policy and task. We use SmolVLA [11], the public LIBERO checkpoint (∼450M parameters: a SmolVLM2 backbone and a flow-matching action expert). We run it in LeRobot with the checkpoint's own settings: 50-step chunks, replanning every step, and 10 Euler steps. We evaluate on LIBERO-Long [12]: 10 tasks, 50 fixed initial states each, and at most 520 steps per episode. Our driver matches LeRobot's `lerobot-eval` success rate on the same task and initial states. In our simulator version the reference policy succeeds in 32% of 500 episodes. That is below the paper's 71%; community reproductions of this checkpoint report 56%.

Compression. **2 flow steps** uses 2 Euler steps instead of 10, with weights unchanged; policy time per call (batch 10, L4) falls from 0.81 s to 0.32 s. **int8** is per-channel round-to-nearest (RTN) weight quantization, and **int4-g32** is RTN int4 with groups of 32. **int4-coarse** is RTN int4 with groups of 128 where the width allows and one group per row otherwise, covering 62% of weights, including every language-model and expert projection. Quantization covers all 521 linear layers. It is simulated (quantize, then dequantize), so it reproduces low-bit numerics but not low-bit speed. We verify that each group holds at most 2^b values.

Monitors. All four are fit once, on reference rollouts.

- **probe-VLM**: a logistic probe on the backbone's mean-pooled final state, with per-step labels from the episode outcome (SAFE-style [4]).
- **probe-expert**: the same probe on the action expert's state at the last denoising step.
- **Mahalanobis**: the distance of the backbone state from successful training steps, using no failure labels (VLA-FAIL-style [6]).
- **action consistency**: the squared disagreement of consecutive chunks on their 49 shared steps (Sentinel-style [7]).

An alarm fires at the first $t \geq 10$ where the running-mean score S_t exceeds τ. The threshold τ is the conformal 0.9 quantile of $\max_{t \geq 10} S_t$ over successful reference calibration episodes. This bounds false alarms on successes at $\alpha = 0.1$ and says nothing about detection.

Protocol. Initial states 0–29 train the monitors (reference only), 30–39 calibrate, and 40–49 test, for every task. All results below are on the test set: 100 episodes per condition, with matched initial states and seeds. Hypotheses, splits and monitors were pre-registered. Everything else is marked exploratory.

3 Results

Step reduction disarms the action-side monitors. With 2 flow steps the policy succeeds as often as the reference, 32% vs. 27%.

- **Action consistency** caught 49% of reference failures (36/73). It now catches none of 68 (Fig. 1b,c). On initial states that fail under both policies, it loses 30 detections and gains none (McNemar $p{=}2{\times}10^{-9}$). The change in detection is -0.49, with a 95% CI of $[-0.91, -0.25]$ when tasks and calibration episodes are resampled together.
- **probe-expert** loses 14 detections and gains none ($p{=}10^{-4}$). Its median alarm moves from step 10 to step 124.
- **Backbone monitors** are unaffected: probe-VLM catches 0.66 vs. 0.67, Mahalanobis 0.32 vs. 0.32. Step reduction changes nothing upstream of the expert. On identical step-0 observations, the backbone state is bit-identical to the reference.[1]

[1] This holds on 6 of 10 tasks, and on 19 of 20 episodes of a seventh. On the other 3, per-process cuDNN autotuning slightly perturbs the reference's step-0 state. We exclude those tasks from same-input comparisons; without them the headline results are unchanged.

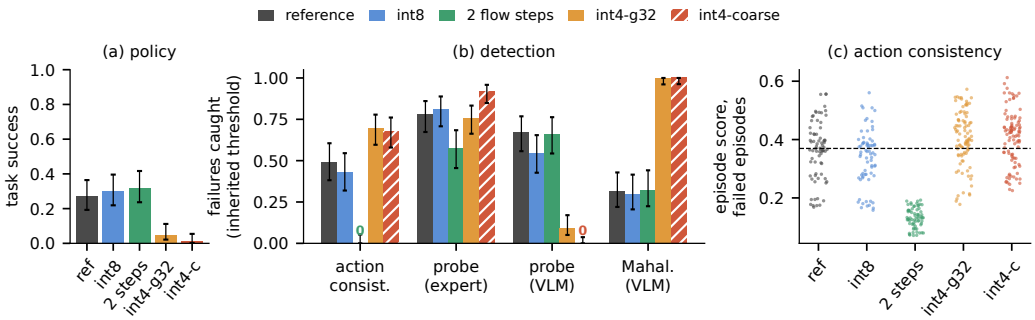


Figure 1: **(a)** Task success. **(b)** The fraction of failed episodes caught with the inherited threshold. Hatched bars are int4-coarse, which collapses the policy. **(c)** Action-consistency scores of failed episodes against the inherited τ (dashed). Error bars are Wilson 95% intervals; the test set has 100 episodes per condition.

Table 1: Reference $\rightarrow$ 2 flow steps on the test set. *Inherited*: the reference-calibrated τ. *Oracle*: the best threshold with FAR $\leq \alpha$, chosen on the test set itself. *Recal.*: τ recalibrated on the accelerated policy's own successful calibration episodes.

	inherited τ		TPR		within-task	recal.
monitor	TPR	FAR	oracle	AUROC	AUROC	TPR/FAR
action consistency	.49→**.00**	.15→.00	.44→.34	.77→.72	.89→.84	.26/.03
probe-expert	.78→**.57**	.11→.00	.77→.71	.95→.94	.98→.89	.66/.06
probe-VLM	.67→.66	.04→.00	.81→.87	.95→.96	.96→.93	.60/.00
Mahalanobis	.32→.32	.07→.06	.36→.38	.85→.86	.78→.78	.29/.06

The failure is in calibration, not information.

- **The score shrinks uniformly.** Two-step sampling shrinks chunk disagreement by a near-constant factor: 0.42 (IQR 0.40–0.44) of the reference on the same initial state, over steps 1–30.
- **Ranking largely survives** (Table 1). AUROC goes from 0.77 to 0.72. An oracle threshold would still catch 34% of failures, against 44% on the reference.
- **Recalibrating** on the accelerated policy's own successes recovers 26% detection at a 3% false-alarm rate.
- **The inherited threshold fails in the reassuring direction.** False alarms fall from 0.15 to 0 for action consistency and from 0.11 to 0 for probe-expert. A team checking its false-alarm budget after acceleration would see an improvement.

Concurrent open-loop work reports this rank-preserving shift for quantized probes [10]. Step reduction shows that it also hits monitors with no learned parameters.

Weight quantization pushes the backbone probe toward "safe".

- **int8** keeps task success (30%) and false-alarm rates within 0.03. Yet on identical inputs it lowers probe-VLM's score in every episode (Table 2). In closed loop the probe catches 0.54 of failures vs. 0.67 (13 lost, 3 gained, p=0.02; exploratory).
- **int4-g32** cuts success to 5% and shifts probe-VLM by -2.3 SD. The probe now catches 9% of failures (39 lost, 0 gained, $p=4\times10^{-12}$).
- **int4-coarse** collapses success to 1% and shifts probe-VLM by -4.6 SD. The probe raises no alarm in any of 99 failed episodes. Mahalanobis alarms on every episode, successes included, so it detects the compression rather than the failures.

An open-loop replay cannot show this case, where the policy itself breaks while its monitor calls it safe.

Table 2: Same-input score shift at step 0 (6 matched tasks, 200 episodes each), in reference standard deviations. In parentheses: the fraction of episodes that moved toward "safe".

monitor	int8	int4-g32	int4-coarse	2 flow steps
probe-VLM	-0.29 (100%)	-2.30 (100%)	-4.58 (100%)	0 (identical)
probe-expert	$+0.09$ (2%)	$+0.05$ (47%)	$+0.81$ (1%)	-1.08 (100%)
Mahalanobis	$+0.05$ (31%)	$+4.82$ (0%)	$+3.98$ (0%)	0 (identical)

Controls.

- **Episode length.** Failures run to the timeout, which can inflate detection [10]. Counting only alarms before step 200 lowers reference detection: probe-VLM .67→.48, action consistency .49→.27. Both zeros stay zero.
- **Task identity.** Two tasks never succeed. Even so, within-task reference AUROC is .96 for probe-VLM and .98 for probe-expert, and excluding those tasks changes no conclusion.
- **Pre-registration.** The primary hypotheses predicted more false alarms under int4. They were untestable, because int4-coarse left one successful test episode. Two secondary hypotheses held: step reduction moves probe-expert more than int8 does, and int8 changes no false-alarm rate by more than 0.05 (AUROC held for 3 of 4 monitors). The detection changes were a pre-registered outcome, but no hypothesis predicted them.

4 Related work and discussion

Related work. VLA monitors are evaluated on the policy they were fit to [4, 6, 13, 7, 14]; SAFE-CAST [5] studies threshold drift under input shift. Language-model probes go stale under fine-tuning but mostly survive quantization by AUROC [9], and concurrent work finds VLA probe coverage shifts under int4 at unchanged AUROC in an open-loop replay [10]. Acceleration studies measure success, latency and motion quality [1, 2, 15, 3], but not the shipped monitor. We add closed-loop policies, step reduction, non-probe monitors, and the one-sided calibration that turns a score shift into silent misses.

Takeaways. Treat any acceleration as invalidating a monitor's calibration, even when success holds. Check detection with labeled failures from the accelerated policy, not only the false-alarm budget. Expect a monitor to break where its input changes: here, backbone monitors survived step reduction and action-side monitors did not.

Limitations. One policy, one suite and one seed, with 100 test episodes per condition. Reference success is low, quantization is simulated, and our monitors are compact versions that use a single threshold where SAFE uses time-varying bands. int4-coarse was an unintended configuration that code review found; we report it under an accurate description.

References

[1] J. Xu, Q. Jin, M. Chen, S. Wang, Y. Sui, and Y. Shi. VLAQuantBench: Closed-loop evaluation of post-training quantization for vision-language-action models. *arXiv preprint arXiv:2609.25376*, 2026.

[2] J. Zhang, Y. Hsieh, Z. Wan, H. Lin, X. Wang, Z. Wang, et al. QuantVLA: Scale-calibrated post-training quantization for vision-language-action models. *arXiv preprint arXiv:2602.20309*, 2026.

[3] R. Liu, T. Zheng, J. Gu, and Z. Wang. CARE: Certifying acceleration for vision-language-action inference. *arXiv preprint arXiv:2610.08917*, 2026.

[4] Q. Gu, Y. Ju, S. Sun, I. Gilitschenski, H. Nishimura, M. Itkina, et al. SAFE: Multitask failure detection for vision-language-action models. *arXiv preprint arXiv:2506.09937*, 2025.

[5] H. Rajaprakash, A. Prajapati, R. Xue, A. Anwar, and J. Thomason. SAFECAST: Robust failure detection for VLA policies with contrast-set training and calibration. *arXiv preprint arXiv:2608.04246*, 2026.

[6] F. Seligmann, E. Gospodinov, E. U. Dincer, and G. Neumann. VLA-FAIL: Efficient task failure detection for finetuned vision-language-action models. *arXiv preprint arXiv:2606.21386*, 2026.

[7] C. Agia, R. Sinha, J. Yang, Z.-a. Cao, R. Antonova, M. Pavone, and J. Bohg. Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress. In *Conference on Robot Learning*, 2024.

[8] A. N. Angelopoulos and S. Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511*, 2021.

[9] E. Duan. Do activation monitors survive model updates? benchmarking, predicting, and repairing activation-monitor staleness. *arXiv preprint arXiv:2606.15980*, 2026.

[10] Anonymous. Rank-preserving, threshold-breaking: Post-training silently decalibrates inherited safety monitors in VLA policies. NeurIPS 2026 workshop submission, OpenReview `atOkXTzFCL`, 2026.

[11] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, et al. SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.

[12] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *NeurIPS Datasets and Benchmarks Track*, 2023.

[13] R. Römer, A. Kobras, L. Worbis, and A. P. Schoellig. Failure prediction at runtime for generative robot policies. *arXiv preprint arXiv:2510.09459*, 2025.

[14] D. Ahmedt-Aristizabal, A. Khamis, M. A. Armin, and L. Petersson. RESCUE: Is this alarm worth acting on? counterfactual evaluation of VLA runtime monitors. *arXiv preprint arXiv:2610.13516*, 2026.

[15] Z. Li, H. Yang, Z. Chen, Y. Chen, et al. From inference efficiency to embodied efficiency: Revisiting efficiency metrics for vision-language-action models. *arXiv preprint arXiv:2603.19131*, 2026.